电商数仓 NL2SQL Agent 项目面试实战场景题集

> *项目名称：安全增强型 NL2SQL Agent*
> *项目代码库：Sammir-z/NL2SQL*
> *核心技术栈：LangGraph / LangChain / FastAPI / MySQL / Qdrant / Elasticsearch / Embedding*
> *说明：本题集结合简历技术亮点与实际项目源码深度提炼，专用于大厂大模型/Agent应用方向的技术面试对齐。答案部分留作讨论占位，后续逐题梳理完善。*

场景一：海量元数据下的取值歧义与召回爆炸（对应技术亮点 1）

题目 1：取值歧义（如“苹果”既是类目又是品牌）如何区分与决策？

- **面试官提问：**
  当用户输入非常口语化且包含多义词时（例如用户问：*“查一下上个月苹果的销售额”*，“苹果”在 ES 中既可能命中 `dim_category.category_name`（水果品类），又可能命中 `dim_brand.brand_name`（数码品牌）），你的取值召回与表组织机制是如何区分并决策的？
- **核心考察点：**
  - ES 全文检索的文档建模与多路召回设计
  - `merge_retrieved_info` 阶段的候选结构合并策略
  - 上下文多阶段消歧（Filter 阶段 vs SQL 生成阶段）
  - 主动追问（Clarification）机制的设计与触发时机
- **【回答记录 / 待完善】：**
  - **底层并存召回：**ES 索引文档为 `(column_id, value)`。检索“苹果”会同时命中两条记录（`dim_category.category_name` 与 `dim_brand.brand_name`）。在 `merge_retrieved_info` 节点中先做**并存保留**，将两张表同时并入候选表集，并将真实取值分别挂载到对应字段的 `examples` 属性中（`category_name.examples = ["苹果"]`，`brand_name.examples = ["苹果"]`）。
  - **上下文多阶段消歧：**

i. `filter_table` 阶段：将用户原始问题与候选表结构（含字段描述和 examples）输入给 LLM。如果问题是“苹果手机”，LLM 会利用自然语言常识剔除 `dim_category`，保留 `dim_brand`。

ii. `generate_sql` 阶段兜底：若问题极度模糊（仅“苹果的销售额”），LLM 会依据数仓事实表指标关联倾向，生成兼容查询（如 `WHERE (brand_name = '苹果' OR category_name = '苹果')`）。

- 生产进阶策略（展现思考）：
  - 主动追问（**Clarification**）机制：计算 ES 召回匹配度的置信度差值，若 Top1 与 Top2 差距小于阈值且缺少上下文修饰，在 Graph 中中断并向前端返回交互单选组件（*“您是指【苹果（品牌）】还是【苹果（生鲜类目）】？”*），由业务人员确认后再生成 SQL，避免无端猜测。

题目 2：候选 Schema 暴增与两路过滤并行的“依赖孤岛”问题

- 面试官提问：
  如果问题命中了多个模糊表或候选字段，导致三路召回合并后的 Schema 信息暴增（Prompt 超长导致上下文溢出或引发 LLM 幻觉），你的 `filter_table` 和 `filter_metric` 节点是如何进行剪枝过滤的？两路过滤并行执行时，会不会出现“保留了指标，却把指标依赖的维表剪掉”的依赖孤岛问题？
- 核心考察点：
  - 大模型只做选择题（输出 JSON 列表）+ 代码硬裁剪的稳定性设计
  - LangGraph 并行分支执行时的数据独立性与依赖冲突
  - 拓扑依赖编排（串行化强依赖锁定）与后置自愈（Self-Healing in `add_extra_context`）
- 【回答记录 / 待完善】：
  - 剪枝稳定性设计（选择题 + 代码硬修剪）：`filter_table` 和 `filter_metric` 提示词要求模型**仅输出结构化的 JSON 数组或键值映射**（只输出保留的表名和字段名列表），真正的 Schema 结构修剪全部由 Python 代码在内存中完成，避免让大模型重写庞大 Schema 带来的格式崩溃与字段漏传，将 Token 消耗降低 60% 以上并大幅抑制幻觉。
  - 并行过滤的“依赖孤岛”风险剖析：

- 在当前编排中，`merge_retrieved_info` 虽补全了指标依赖的 `relevant_columns`，但由于 `filter_table` 与 `filter_metric` 是并行分支，极端情况下可能出现：`filter_metric` 保留了“GMV 指标”，而 `filter_table` 的 LLM 产生幻觉剔除了 GMV 依赖的订单表或维表，导致后续 `generate_sql` 上下文断裂。

- **生产级解决方案（拓扑依赖编排 & 后置自愈）：**

  i. **拓扑串行化与强依赖锁定（推荐）：**将编排调整为 `filter_metric` 先执行，输出最终选中的指标，并将其 `relevant_columns` 所属的表/字段打上“强依赖白名单（Pinned Schema）”，作为硬约束喂给 `filter_table`，`filter_table` 仅允许剔除游离无依赖的字段。

  ii. **后置自愈守门员（Self-Healing in `add_extra_context`）：**在进入 `generate_sql` 前的 `add_extra_context` 节点增加 Schema 完整性断言，若发现选中指标依赖的字段未在 `table_infos` 中，自动从 MySQL 元数据兜底重新加载补齐。

场景二：长链路多阶段编排的端到端时延（P95）优化（对应技术亮点 2）

题目 1：端到端耗时拆解与压缩至 3 秒以内的重构策略

- **面试官提问：**
  在你的 LangGraph 编排中，整条链路下来至少涉及 3~4 次大模型调用以及多次向量/全文/DB 检索。在真实业务交互下，端到端耗时通常较高。你在项目中是否有做过时延剖析？如果要求把端到端 P95 响应时间从十几秒压缩到 3 秒以内，从 Graph 图编排和模型调用策略上你会做哪些重构或合并？

- **核心考察点：**
  - 各节点耗时分解能力（jieba 2ms vs 扩词 1s vs 过滤 2s vs 生成 3s）
  - 减少串行 LLM 调用（规则/同义词替代 LLM 扩词、向量分值阈值跳过过滤节点）
  - 模型分级（Model Tiering，小模型快速过滤 + 大模型核心生成）
  - 多级元数据缓存与 SQL 语义缓存（Semantic Cache）

- **【回答记录 / 待完善】：**
  - **耗时分布剖析（Profiling）：**

- `extract_keywords`：采用本地 `jieba` TF-IDF，**耗时仅 2~5ms**，纯本地计算无瓶颈。
- `recall_value`：调用了 1 次 LLM 进行扩词（`extend_keywords_for_value_recall`），耗时约 **1.0~1.5s**；ES 检索本身仅需 **10~20ms**。
- `recall_column` & `recall_metric`：并行 Embedding + Qdrant 检索，耗时约 **150~300ms**。
- `filter_table` & `filter_metric`：2 次并行 LLM 调用，耗时取决于较慢者，约 **1.5~2.5s**。
- `generate_sql`：核心生成任务，输出长 SQL，耗时约 **2.0~3.5s**。
- `guard_sql` & `validate_sql`：`sqlglot` AST 解析约 **5ms**，MySQL `EXPLAIN` 校验约 **15ms**。
- **基线总结**：全流程首次命中端到端 P95 约 **5~8 秒**；若触发 `correct_sql` 重试，耗时将超过 **10 秒**。

- **压缩至 3s 以内的 4 大重构策略：**

  i. **砍掉冗余串行 LLM 调用：**

  - 移除 `recall_value` 的 LLM 扩词节点，改用本地分词 + 业务同义词词典 + ES 的 Pinyin/N-gram/Fuzzy Query 模糊匹配，直接节省 **1.0~1.5s**。
  - 引入向量打分阈值快速通路：对于候选表 ≤ 3 且向量置信度 > 0.75 的常规查询，直接跳过 `filter_table` 和 `filter_metric`，直通 `generate_sql`。

  ii. **模型分级（Model Tiering）**：过滤节点改用轻量级小模型（如 Qwen-2.5-7B + vLLM 部署），并发过滤时延压到 300ms 内；仅核心生成使用高推理能力模型。

  iii. **多级元数据与语义缓存（Semantic Cache）：**

  - 基于向量相似度（如 GPTCache / Redis）对高频分析问题建立 SQL 缓存，命中相似度 > 0.95 时直接返回历史验证过的 SQL，实现 **50ms** 极速响应。
  - 元数据全量内存预热（Redis / Local Cache），消除图节点中的 MySQL I/O 阻塞。

  iv. **流式首字体验优化**：利用 LangGraph 的 `stream_mode="custom"` 在生成阶段流式输出 Token，使前端在 1 秒左右看到控制台状态更新与思考过程，降低体感等待。

题目 2：Qdrant / ES 检索超时抖动时的容错与降级策略

- **面试官提问：**
  如果 Qdrant 节点偶发超时抖动，或者 ES 检索阻塞，当前图工作流会直接失败还是有降级降效策略（Graceful Degradation）？
- **核心考察点：**
  - 异步超时控制与熔断机制（`asyncio.wait` 与 timeout 设置）
  - 局部降级策略：ES 失败降级为字段宽泛匹配，Qdrant 失败降级为 DB 精确匹配或核心宽表兜底
  - 降级标记在 Graph 状态中的流转与前端透明提示
- **【回答记录 / 待完善】：**
  - **异步超时熔断控制**：在调用 Qdrant 与 ES 时配置严格的客户端超时（如设置 `timeout=300ms`），配合 `asyncio.wait(..., return_when=ALL_COMPLETED)`，超时即熔断，避免单一组件拖垮整图生命周期。
  - **各节点差异化降级策略（Fail-Safe）：**
    - **ES（字段值）超时降级**：取值仅用于丰富枚举条件。ES 故障时捕获异常将 `retrieved_value_infos` 设为空列表并放行。LLM 虽缺失具体枚举示例，但仍可根据字段名（如 `region_name`）生成宽松条件（如 `LIKE '%华北%'`），主流程不中断。
    - **Qdrant（向量召回）超时降级**：向量不可用时，自动降级至 Meta MySQL 的全文搜索/模糊匹配（表名、字段名、指标名）；若全量召回失效，触发**核心底表兜底机制（Fallback Schema）**，直接加载数仓高频宽表（如 `dws_trade_shop_daily_summary`）保证核心查询可用。
  - **降级状态透明化**：在 State 中注入 `is_degraded: True` 和 `degraded_reason`，一方面在下游 `generate_sql` 的 Prompt 中提示模型“当前维表枚举不可用，请使用标准宽松匹配”，另一方面通过 SSE 向前端流式提示：“部分维表索引响应较慢，已为您启用快速降级查询”，保证用户知情权。

场景三：“语法合法但业务算错”与纠错闭环边界（对应技术亮点 3、5）

题目 1：EXPLAIN 校验的盲区与“隐性业务错误”的守门机制

- **面试官提问：**
  `EXPLAIN` 只能捕获静态语法错误、表/字段不存在等底层异常。如果模型生成的 SQL 语法完全合法（`EXPLAIN` 顺利通过），但业务逻辑算错了（例如 GMV 算成了含退款金额、去重漏了 `DISTINCT`、JOIN 产生了笛卡尔积），这种“隐性业务错误”你的校验闭环能发现吗？生产环境中该如何设计守门机制？
- **核心考察点：**
  - 静态 SQL EXPLAIN 校验的本质与局限性
  - 指标元数据中的业务口径规范（计算公式、过滤条件强约束）
  - 数据质量规则校验（空值检查、数值范围与量级合理性预检、行数暴增预警）
  - LLM-as-a-Judge / 业务语义反思校验机制
- **【回答记录 / 待完善】：**
  - **坦诚界定 EXPLAIN 的本质与盲区：**
    - `EXPLAIN` 本质上是数据库优化器的静态语法编译与执行计划生成，能拦截拼写错误、类型转换失败、聚合缺失（`ONLY_FULL_GROUP_BY`）、无权限等底层异常。
    - 但在数据库眼里，**算错业务口径的 SQL 和正确的 SQL 同样是语法合法的**（例如漏了 `WHERE order_status != 'REFUNDED'`、统计用户漏了 `DISTINCT`、或两表关联无键产生笛卡尔积），`EXPLAIN` 无法感知此类隐性业务偏差。
  - **生产级“四道防线”守门机制设计：**
    i. **防线一：指标语义层（Semantic Layer）强注入（执行前）：**元数据库不仅维护指标文字描述，更固化其标准 SQL 片段与强制过滤条件（如 `gmv` 强绑定 `SUM(pay_amount)` 与 `status = 1`）。模型生成时必须原样复用公式，消除自由发散空间。
    ii. **防线二：执行计划代价深度嗅探（执行前）：**解析 `EXPLAIN` 详细结果，若 `Extra` 包含 `Using join buffer (Cartesian product)` 或 `type = ALL` 且扫描行数超过阈值（如百万行）且未命中时间分区键，直接判定为致命错误熔断拦截。

iii. **防线三：数据质量断言预检（执行后）**：在流式返回前增加规则过滤：比率指标（转化率/毛利率）超出 [0, 1] 范围、聚合指标单行预期返回数万行（GROUP BY 粒度失控）、关键列计算全为 NULL 等，均触发异常告警。

iv. **防线四：轻量级 LLM 反思交叉审计（LLM-as-a-Judge）**：针对高风险或大宽表，将用户原始问题、生成 SQL 及前 3 行执行结果送入反思节点，快速判断业务意图是否准确闭环。

题目 2：纠错循环中的“震荡退化 / 负向修正”与防退化设计

- **面试官提问：**
  在纠错闭环中，如果模型在 `correct_sql` 时产生了“修了 A 却破坏了 B”的震荡退化现象（比如修好了语法报错，但顺手把原问题中的时间范围过滤条件改丢了），你的 Prompt 和状态机是如何防止这种“负向修正”的？
- **核心考察点：**
  - 纠错 Prompt 的上下文锁定原则（保留完整原始 Context 与问题约束）
  - 最小修改原则（Minimal Edit Principle）
  - 状态机多轮历史记录追踪与退化检测
  - `max_sql_retries` 边界控制与熔断失败策略
- **【回答记录 / 待完善】：**
  - **退化成因剖析**：模型看到数据库报错（如 `Unknown column 'user_name'`）时注意力发生局部过度拟合，全局上下文注意力漂移，重写时容易误删已有的时间约束或破坏原有关联条件；另外也容易陷入改了 A 错报 B 错的改错死循环。
  - **项目中的已有防御设计：**
    i. **全局上下文持久锚定**：在 `correct_sql.py` 中，修正节点全量注入了 `table_infos`, `metric_infos`, `date_info`, `db_info`, `query`，保证模型始终具备全局业务视图。

    ii. **最小必要修改原则（Minimal Edit Principle）**：在 `correct_sql.prompt` 中明确强约束：“必须严格基于错误修正，不得无依据重写 SQL”、“严格保持原始业务语义不变，不得擅自新增、删除或替换过滤条件与时间范围”。

    iii. **最大重试硬熔断**：通过 `state["validation_attempt"] <= state["max_sql_retries"]` 控制退出边界，防止无限死循环消耗 Token。
  - **生产进阶防退化策略：**

- **AST 语法树语义 Diff 拦截（AST Semantic Diff）**：利用 `sqlglot` 提取纠错前后的 WHERE 约束子树。如果纠错后丢失了原有的时间区间或核心过滤常量，直接阻断回滚，并向模型二次提示：“检测到时间过滤条件被误删，请在原 SQL 基础上仅修剪局部语法错误！”
- **死锁震荡检测（Oscillation Detection）**：在 State 中维护 `error_history`。若出现相同错误重复出现或在两种错误间横跳，判定为逻辑死锁，立即主动熔断转入 `fail_query`，停止盲目试错。

场景四：SQL 语法树（AST）安全防御与多租户行级注入（对应技术亮点 3）

题目 1：多层子查询 / CTE / UNION ALL 下的行级租户过滤注入

- **面试官提问：**
  在你的 `SQLGuard` 中，使用 `sqlglot` 对 AST 树进行了精细控制。假设生成的 SQL 包含复杂的多层嵌套子查询、CTE（WITH 语句）或者 UNION ALL，如果某个受控租户表藏在子查询内部，当前的 `_inject_row_filters` 是如何确保租户过滤条件（如 `tenant_id IN (...)`）被百分之百注入到底层子查询，而不是挂在外层导致全表数据穿透？
- **核心考察点：**
  - `sqlglot` AST 树遍历机制（`statement.find_all(exp.Table)` 与作用域 Scope 解析）
  - 嵌套查询与 CTE 视图中的表别名及作用域穿透
  - 统一行级安全过滤（Row-Level Security, RLS）注入算法
- **【回答记录 / 待完善】：**
  - **现有实现的风险剖析（根节点挂载局限）：**
    - 当前 `_inject_row_filters` 遍历了整棵语法树的所有 `Table`，但统一调用 `statement.set("where", ...)` 将谓词挂载在根 `Select` 节点上。
    - **风险 1（语法报错）**：当遇到 Derived Table 子查询（如 `SELECT * FROM (SELECT order_id, tenant_id FROM fct_order) t`），外层只认别名 `t`，直接挂载 `fct_order.tenant_id = 'xxx'` 会报 `Unknown column` 错误。

- 风险 2（越权穿透）：在 `UNION ALL` 复合查询中，外层挂 WHERE 无法渗透到所有独立联合分支，导致未注入的分支读取到全量租户数据。

- 生产级解决方案（基于 **Scope 作用域就地注入**）：

  i. `sqlglot` 作用域遍历（**Scope-aware Injection**）：

  - 利用 `sqlglot.optimizer.scope.traverse_scope(statement)` 遍历每个独立的作用域（包括 CTE 内部、Derived Table 子查询、UNION 的各 Select 分支）。
  - 对每个 Scope，只查找其直接引用的物理表（过滤掉上层别名或 CTE 引用），在该 Scope 自身的 WHERE 子句中**就地注入**对应的租户条件（`exp.In(exp.column(tenant_id, table=alias), ...)`）。
  - **收益**：既保证了语法绝对合规，又让过滤条件下沉到最底层子查询，提前截断内层全表扫描。

  ii. 底层存储引擎行级安全（**Row-Level Security, RLS**）双保险：

  - 在数仓底层（如 PostgreSQL / MySQL 视图或存储引擎策略）配置 RLS，根据请求 Token 中的 `tenant_id` 自动约束底层行可见性，形成“应用层 AST 改写 + 数据库内核安全策略”的双重防护网。

题目 2：针对 Prompt Injection 的 AST 防御与盲注危险函数拦截

- **面试官提问：**
  如果恶意用户通过 Prompt Injection 诱导大模型写出类似 `SELECT * FROM (SELECT benchmark(10000000, md5('abc'))) t` 或利用字符串拼接绕过函数黑名单，AST 校验层面你还有哪些加固措施？
- **核心考察点：**
  - 黑名单机制的脆弱性与白名单机制升级
  - 危险函数、系统内置表、元数据信息表（`information_schema` 等）拦截
  - 只读事务、查询超时（Query Timeout）、Max Rows 限制等数据库层兜底防御
- **【回答记录 / 待完善】：**

- **黑名单机制的脆弱性：**`_BLOCKED_FUNCTIONS` 黑名单无法穷举所有攻击手段，攻击者可通过未收录的系统函数（如高迭代 `JSON_STORAGE_SIZE`）、冷门加密哈希函数、或 `CHAR(...)` 动态编码绕过关键字过滤。
- **生产级纵深防御体系（Defense in Depth）：**

    i. **函数与系统表纯白名单机制（Strict Whitelist）：**

        - **函数白名单：**AST 遍历所有 `exp.Anonymous` 和 `exp.Func` 节点，强制校验必须属于数仓分析白名单（`SUM, AVG, COUNT, MIN, MAX, DATE_FORMAT, DATEDIFF, COALESCE, ROUND, IF, CONCAT` 等），未授权函数一律拦截。
        - **系统表空间隔离：**严格要求 AST 中的 Table 名必须在受控业务元数据白名单内，坚决阻断任何访问 `information_schema.*`、`mysql.*`、`performance_schema.*`、`sys.*` 的元数据探测行为。

    ii. **AST 结构级硬性约束：**

        - **单语句与只读断言：**根节点必须为 `exp.Select`，且解析结果长度必须为 1，杜绝分号多语句注入；禁止 `INTO OUTFILE/DUMPFILE` 写文件提权及任何 DDL/DML 操作。
        - **强制 LIMIT 覆盖：**代码已有 `_apply_limit`，无条件注入或改写 `LIMIT <= max_result_rows`，防止恶意巨量查询打爆内存。

    iii. **基础设施底座硬兜底：**

        - **数仓只读账号（Read-Only User）：**连接池使用无写入权限的数据库账号，从权限层杜绝写操作。
        - **会话级执行超时熔断：**数据库连接强制注入 `SET max_execution_time = 3000`（3 秒超时硬熔断）。即使攻击者构造了高消耗死循环，MySQL 内核也会在 3 秒时强行 KILL QUERY。

场景五：流式接口与并发链路追踪（对应技术亮点 4）

题目 1：LangGraph 并行分支与异步线程池下的 ContextVar 上下文传播

- **面试官提问：**
  你在 HTTP 中间件中使用 `ContextVar` 存储 `request_id` 串联并发日志。在 LangGraph 的并行分支（例如三路召回同时并发执行）以及异步线程池（如 `run_in_executor`）执行过程中，`ContextVar` 的上下文传播机制是怎样的？会不会发生跨协程的上下文丢失或污染？
- **核心考察点：**
  - Python `asyncio` 中 `ContextVar` 在 `create_task` 时的浅拷贝（Shallow Copy）语义
  - 线程池 `run_in_executor` 丢失上下文的典型坑点与 `contextvars.copy_context().run` 解决方案
  - 日志模块 Logger Filter 对 ContextVar 的提取与格式化
- **【回答记录 / 待完善】：**
  - **协程并发下的传播机制（Copy-on-Write 浅拷贝）：**
    - 当请求进入 FastAPI 中间件时，生成全局唯一 `request_id` 并 `request_id_ctx_var.set(request_id)`。
    - 在 LangGraph 的并行分支（如 `recall_column`, `recall_value`, `recall_metric` 通过 `asyncio.gather` 并发调度）中，Python `asyncio.create_task` 底层**自动对当前上下文执行浅拷贝（Shallow Copy）**。
    - 各并行分支天然继承了同一个 `request_id`，Loguru 通过 `logger.patch(inject_request_id)` 能精准提取并打印，无需手动传递参数；且因为写时复制（Copy-on-Write）特性，互不污染。
  - **经典坑点与跨线程池穿透（Thread Pool 陷阱）：**
    - **问题**：若将 CPU 密集型任务（如 jieba 分词、复杂 AST 解析）放入 `loop.run_in_executor(None, sync_func)` 执行，原生操作系统线程**默认不继承协程上下文**，导致 `ContextVar` 跌落为默认值（如 `"1"`），日志链路断裂。
    - **生产解决方案：**

i. 使用现代标准方法（Python 3.9+）：`asyncio.to_thread(sync_func, *args)`，其内部自动捕获并传递当前 `contextvars.copy_context()`；

ii. 自定义线程池显式传递：`ctx = contextvars.copy_context()`，再调用 `await loop.run_in_executor(pool, ctx.run, sync_func, *args)`。

题目 2：客户端连接断开时的取消信号传播（Cancellation Propagation）

- **面试官提问：**
  在 SSE 流式输出场景下，如果业务人员点击了查询后立即关闭浏览器页面或刷新，客户端 TCP 连接断开。此时后端的 LangGraph 图任务、远端大模型生成以及数仓查询是会继续空转消耗算力，还是会被立即 Cancel？你是怎么做取消传播的？
- **核心考察点：**
  - FastAPI / Starlette `StreamingResponse` 与客户端断开检测（`request.is_disconnected()`）
  - 异步生成器（Async Generator）的取消传播与 `CancelledError` 捕获
  - 对外 API 调用（LLM Client）与数仓长查询（MySQL `KILL QUERY`）的资源及时回收
- **【回答记录 / 待完善】：**
  - **断连传播现状与盲区剖析：**
    - **ASGI / Python 协程层：**客户端断开 TCP 连接时，Uvicorn 触发 `http.disconnect`，Starlette 的 `StreamingResponse` 会向正在执行的异步生成器抛出 `asyncio.CancelledError`（Python 3.8+ 继承自 `BaseException`，不会被业务代码的 `except Exception` 误吞），协程任务能被正常打断，远端 HTTP 客户端（如 httpx / aiohttp）会随之断开连接，避免继续接收 Token。
    - **数仓层空转盲区：**若当前正处于 `run_sql` 节点执行复杂全表扫描，Python 侧即使中断，**MySQL 服务端仍在后台继续执行该 SQL 直至结束**，造成数仓计算资源严重浪费。
  - **生产级全链路取消闭环（Cancellation Propagation）：**

i. **心跳与主动断开探测**：将 `Request` 透传至数据层，在耗时循环与流式 yield 节点显式加入 `if await request.is_disconnected(): raise asyncio.CancelledError()`，提前检测断连。

ii. **数仓查询强行熔断（`KILL QUERY`）**：

- 在执行数仓查询前，通过底层数据库连接获取当前会话的 `CONNECTION_ID()`（或线程 ID）存入上下文；
- 在异常处理与 `finally` 块中捕获 `asyncio.CancelledError`，从独立的管理连接池异步触发 `KILL QUERY <connection_id>`，从数据库内核层面强制中止慢查询，彻底终结算力空转。

场景六：Evaluation 评测体系与实战落地（对应技术亮点 5）

题目 1：50 条测试集的客观性与“86% 首次错误修复率”背后的真实归因

- **面试官提问：**
  简历中提到“在 50 条测试问题上，将 SQL Execution Accuracy 从 72% 提升至 78%，其中 86% 的首次生成错误可通过自动 Repair 修复”。这 50 条测试集的数据分布是怎样的？那 14% 最终无法被 Repair 修复的案例，根本原因是什么？
- **核心考察点：**
  - 评估集设计的难度分级（简单单表、多表 JOIN、指标计算、复杂嵌套）
  - 评测指标设计（`canonical_rows` 结果集对齐 vs Exact Match）
  - Repair 成功类型的归因（表名别名错误、缺失 GROUP BY、MySQL 版本语法差异等易修问题）
  - 失败 Case 归因（元数据召回缺失导致的信息不足、复杂的业务多表 JOIN 拓扑混乱）
- **【回答记录 / 待完善】：**

> *（待讨论填充）*